



Cyberscope

A *TAC Security* Company

Audit Report

VaultedMetrics Token

February 2026

SHA256 :

4e5798571e40f828789ee7643acf563fa15f2b5a47c6ebe727bce49097a8b1d3

Audited by © cyberscope

Table of Contents

Table of Contents	1
Risk Classification	3
Review	4
Audit Updates	4
Source Files	4
Overview	5
VM888Cyberscope.sol	5
Findings Breakdown	6
Diagnostics	7
ST - Stops Transactions	9
Description	9
Recommendation	9
BC - Blacklists Addresses	11
Description	11
Recommendation	11
IBP - Inaccurate Buffer Progression	13
Description	13
Recommendation	14
CCR - Contract Centralization Risk	15
Description	15
Recommendation	16
IBC - Inconsistent Block Calculation	17
Description	17
Recommendation	18
MV - Misleading Variables	19
Description	19
Recommendation	19
MEE - Missing Events Emission	20
Description	20
Recommendation	21
NWES - Nonconformity with ERC-20 Standard	22
Description	22
Recommendation	22
PLPI - Potential Liquidity Provision Inadequacy	23
Description	23
Recommendation	24
PMRM - Potential Mocked Router Manipulation	25
Description	25
Recommendation	25

PTRP - Potential Transfer Revert Propagation	26
Description	26
Recommendation	26
PVC - Price Volatility Concern	27
Description	27
Recommendation	28
RCS - Redundant Conditional Statements	29
Description	29
Recommendation	29
SCB - Sell Cooldown Bypass	30
Description	30
Recommendation	31
UPR - Unprotected Price Reading	32
Description	32
Recommendation	34
L03 - Redundant Statements	35
Description	35
Recommendation	35
L16 - Validate Variable Setters	36
Description	36
Recommendation	37
L19 - Stable Compiler Version	38
Description	38
Recommendation	38
Functions Analysis	39
Inheritance Graph	43
Flow Graph	44
Summary	45
Disclaimer	46
About Cyberscope	47

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Audit Updates

Initial Audit	16 Feb 2026
---------------	-------------

Source Files

Filename	SHA256
VM888Cyberscope.sol	4e5798571e40f828789ee7643acf563fa15f2b5a47c6ebe727bce49097a8b1d3

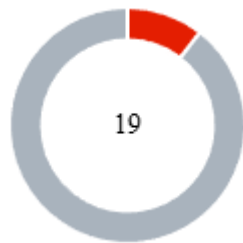
Overview

VM888Cyberscope.sol

Vaulted Metrics Token is an ERC-20 token with integrated Reflect Finance (RFI) mechanics, where non-excluded holders passively earn reflections through a shrinking reflected supply (`_rTotal`). The fixed supply of 888,000,000 tokens is minted entirely to the deployer at construction. Every transfer routes through `_transfer()`, which enforces pause state, blacklist checks, trading activation, per-transaction and per-wallet caps, and a 30-minute sell cooldown. Transfers are classified as buys, sells, or wallet-to-wallet, each subject to distinct tax schedules comprising reflections, auto-LP, treasury, and staking components, with additional dynamic loyalty and whale dump surcharges on sells. An anti-snipe mechanism applies a 99% tax during the first blocks after trading is enabled.

Tax tokens accumulate in the contract and are periodically swapped to ETH via `_swapAndDistribute()`, which adds permanent liquidity (LP tokens burned to `0xdead`), distributes ETH to treasury and staking wallets, and funds a buyback reserve. An autonomous buyback engine monitors the spot price against a lazily-updated 30-day EMA; when price falls below EMA, the contract buys and burns tokens. The contract is owner-controlled with administrative functions for blacklisting, exclusions, pause control, and token recovery. `renounceOwnership()` automatically unpauses and permanently disables all admin functions.

Findings Breakdown



● Critical	2
● Medium	0
● Minor / Informative	17

Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	0	2	0	0
● Medium	0	0	0	0
● Minor / Informative	4	13	0	0

Diagnostics

● Critical ● Medium ● Minor / Informative

Severity	Code	Description	Status
●	ST	Stops Transactions	Unresolved
●	BC	Blacklists Addresses	Unresolved
●	IBP	Inaccurate Buffer Progression	Unresolved
●	CCR	Contract Centralization Risk	Unresolved
●	IBC	Inconsistent Block Calculation	Unresolved
●	MV	Misleading Variables	Unresolved
●	MEE	Missing Events Emission	Unresolved
●	NWES	Nonconformity with ERC-20 Standard	Unresolved
●	PLPI	Potential Liquidity Provision Inadequacy	Unresolved
●	PMRM	Potential Mocked Router Manipulation	Unresolved
●	PTAI	Potential Transfer Amount Inconsistency	Unresolved
●	PTRP	Potential Transfer Revert Propagation	Unresolved
●	PVC	Price Volatility Concern	Unresolved
●	RCS	Redundant Conditional Statements	Unresolved

●	SCB	Sell Cooldown Bypass	Unresolved
●	UPR	Unprotected Price Reading	Unresolved
●	L03	Redundant Statements	Unresolved
●	L16	Validate Variable Setters	Unresolved
●	L19	Stable Compiler Version	Unresolved

ST - Stops Transactions

Criticality	Critical
Location	VM888Cyberscope.sol#L1075
Status	Unresolved

Description

The contract owner has the authority to stop the transactions for all users. The owner may take advantage of it by calling the `pause()` function. Additionally, transactions are initially disabled for all accounts not excluded from tax by the owner.

Furthermore, the contract enforces a high anti-snipe tax during the initial blocks after trading is enabled, which effectively penalizes any non-excluded address attempting to trade during this period. Sales from non-excluded addresses are also restricted by a cooldown period of 30 minutes throughout the life-cycle of the contract.

```
Shell
function pause() external onlyOwner {
    paused = true;
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

BC - Blacklists Addresses

Criticality	Critical
Location	VM888Cyberscope.sol#L1057
Status	Unresolved

Description

The contract owner has the authority to stop addresses from transactions. The owner may take advantage of it by calling the `setBlacklist` function.

```
Shell
function setBlacklist(address account, bool blacklisted)
external onlyOwner {
    if (owner() == address(0)) revert
    CannotBlacklistAfterRenounce();
    isBlacklisted[account] = blacklisted;
    emit BlacklistUpdated(account, blacklisted);
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.

- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

IBP - Inaccurate Buffer Progression

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L968,980
Status	Unresolved

Description

The contract maintains a 7-day circular buffer to track daily reflections for APY estimation. The buffer advances one position each time a day boundary is detected during a transaction. However, if multiple days pass without any transaction activity, the buffer only advances a single position rather than the actual number of elapsed days. As a result, stale data from previous cycles remains in the uncleared buffer positions and is included in the APY calculation. The `getAPY()` function always divides the total by 7, producing inaccurate APY values when the buffer contains outdated entries from periods outside the intended 7-day window.

```
Shell
function _checkDayBoundary() private {
    if (block.timestamp >= lastDayRecorded + 1
days) {
        dailyReflections[currentDayIndex] =
todayReflections;
        currentDayIndex = (currentDayIndex + 1) %
APY_WINDOW;
        todayReflections = 0;
        lastDayRecorded = block.timestamp;
    }
}

function getAPY() external view returns (uint256)
{
    uint256 totalReflections7Days;
    for (uint256 i = 0; i < APY_WINDOW; i++) {
```

```
        totalReflections7Days +=  
dailyReflections[i];  
    }  
    uint256 avgDailyReflections =  
totalReflections7Days / APY_WINDOW;  
    ...  
    return (avgDailyReflections * 365 * 10000) /  
tCirculating;  
}
```

Recommendation

The team is advised to ensure that the buffer progression accounts for the actual number of elapsed days, clearing any skipped positions accordingly. This will maintain the accuracy of the reported APY regardless of transaction frequency.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L1014,1020,1044,1057,1064,1069,1075,1080,11021093,
Status	Unresolved

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

Shell

```
function depositBuybackETH() external payable onlyOwner {

function excludeFromReflections(address account) external
onlyOwner {

function includeInReflections(address account) external
onlyOwner {

function enableTrading() external onlyOwner {

function setBlacklist(address account, bool blacklisted)
external onlyOwner {

function setExcludedFromTax(address account, bool excluded)
external onlyOwner{

function setExcludedFromLimits(address account, bool
excluded) external onlyOwner {

function pause() external onlyOwner {
```

```
function unpause() external onlyOwner {  
  
function recoverERC20(address tokenAddress, uint256 amount)  
external onlyOwner {  
  
function recoverETH(uint256 amount) external onlyOwner {
```

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

IBC - Inconsistent Block Calculation

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L204,534
Status	Unresolved

Description

The contract implements an anti-snipe mechanism that applies a 99% tax during the first blocks after trading is enabled. The constant `ANTI_SNIPE_BLOCKS` is set to 2, suggesting that the protection should cover 2 blocks. However, the conditional statement uses a less-than-or-equal operator (`<=`) instead of strict less-than (`<`). As a result, the anti-snipe tax is applied for 3 blocks instead of the expected 2, creating an inconsistency between the constant naming and the actual behavior.

```
Shell
uint256 public constant ANTI_SNIPE_BLOCKS = 2;

...

if (block.number <= tradingEnabledBlock +
    ANTI_SNIPE_BLOCKS) {
    taxes.autoLPFee = (amount *
        ANTI_SNIPE_TAX_BPS) / 10000;
    taxes.totalFee = taxes.autoLPFee;
    return taxes;
}
```

Recommendation

The team is advised to ensure that the anti-snipe duration is consistent with the constant naming and intended documentation. The conditional operator should accurately reflect the desired number of protected blocks to prevent any discrepancy between expected and actual behavior.

MV - Misleading Variables

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L174,182,189,800
Status	Unresolved

Description

Variables can have misleading names if their names do not accurately reflect the value they contain or the purpose they serve. The contract uses some variable names that are too generic or do not clearly convey the information stored in the variable. Misleading variable names can lead to confusion, making the code more difficult to read and understand.

```
Shell
address public constant STAKING_WALLET =
0x40bA7dB618BD9eE95E9745D598629c0fF69c84eE;

uint256 public constant BUY_STAKING_BPS = 300;

uint256 public constant SELL_STAKING_BPS = 300;
...

(bool stakingSuccess,) =
payable(STAKING_WALLET).call{value: ethForStaking}("");
```

Recommendation

It's always a good practice for the contract to contain variable names that are specific and descriptive. The team is advised to keep in mind the readability of the code.

MEE - Missing Events Emission

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L1064,1069,1075,1080
Status	Unresolved

Description

The contract performs actions and state mutations from external methods that do not result in the emission of events. Emitting events for significant actions is important as it allows external parties, such as wallets or dApps, to track and monitor the activity on the contract. Without these events, it may be difficult for external parties to accurately determine the current state of the contract.

Shell

```
function setExcludedFromTax(address account, bool excluded)
external onlyOwner
```

```
function setExcludedFromLimits(address account, bool
excluded) external onlyOwner
```

```
function pause() external onlyOwner {
```

```
function unpause() external onlyOwner {
```

Recommendation

It is recommended to include events in the code that are triggered each time a significant action is taking place within the contract. These events should include relevant details such as the user's address and the nature of the action taken. By doing so, the contract will be more transparent and easily auditable by external parties. It will also help prevent potential issues or disputes that may arise in the future.

NWES - Nonconformity with ERC-20 Standard

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#LL434
Status	Unresolved

Description

The contract does not fully conform to the ERC20 Standard. Specifically, according to the standard, transfers of 0 values must be treated as normal transfers and fire the Transfer event. However, the contract implements a conditional check that prohibits transfers of 0 values. This discrepancy between the contract's implementation and the ERC20 standard may lead to inconsistencies and incompatibilities with other contracts.

```
Shell
function _transfer(address from, address to,
uint256 amount) private {
    ...

    if (amount == 0) revert
TransferAmountZero();
```

Recommendation

The incorrect implementation of the ERC20 standard could potentially lead to problems when interacting with the contract, as other contracts or applications that expect the ERC20 interface may not behave as expected. The team is advised to review and revise the implementation of the transfer mechanism to ensure full compliance with the ERC20 standard. <https://eips.ethereum.org/EIPS/eip-20>.

PLPI - Potential Liquidity Provision Inadequacy

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L820,837,862
Status	Unresolved

Description

The contract operates under the assumption that liquidity is consistently provided to the pair between the contract's token and the native currency. However, there is a possibility that liquidity is provided to a different pair. This inadequacy in liquidity provision in the main pair could expose the contract to risks. Specifically, during eligible transactions, where the contract attempts to swap tokens with the main pair, a failure may occur if liquidity has been added to a pair other than the primary one. Consequently, transactions triggering the swap functionality will result in a revert.

Shell

```
function _swapTokensForETH(uint256 tokenAmount) private {  
    ...  
  
    uniswapRouter.swapExactTokensForETHSupportingFeeOnTransferT  
    okens(  
  
        function _addLiquidity(uint256 tokenAmount, uint256  
        ethAmount) private {  
            ...  
            uniswapRouter.addLiquidityETH{value: ethAmount}(  
  
            uniswapRouter.swapExactETHForTokensSupportingFeeOnTransferT  
            okens{value: ethToSpend}
```

Recommendation

The team is advised to implement a runtime mechanism to check if the pair has adequate liquidity provisions. This feature allows the contract to omit token swaps if the pair does not have adequate liquidity provisions, significantly minimizing the risk of potential failures.

Furthermore, the team could ensure the contract has the capability to switch its active pair in case liquidity is added to another pair.

Additionally, the contract could be designed to tolerate potential reverts from the swap functionality, especially when it is a part of the main transfer flow. This can be achieved by executing the contract's token swaps in a non-reversible manner, thereby ensuring a more resilient and predictable operation.

PMRM - Potential Mocked Router Manipulation

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L312,318
Status	Unresolved

Description

The contract accepts the router address as an argument during deployment. While this feature provides flexibility, it introduces a security threat. The owner could set the router address to any contract that implements the router's interface, potentially containing malicious code. Since `uniswapRouter` is declared as immutable, this address cannot be corrected after deployment. In the event of a transaction triggering the swap functionality with such a malicious contract as the router, all internal operations including auto-swaps, liquidity additions, and buybacks may be manipulated.

```
Shell
constructor(address _router) Ownable(msg.sender) {
    ...
    IUniswapV2Router02 router =
    IUniswapV2Router02(_router);

    uniswapRouter = router;
```

Recommendation

The team should consider including the router address as a constant immutable value in the codebase.

PTRP - Potential Transfer Revert Propagation

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L793,800
Status	Unresolved

Description

The contract sends funds to `TREASURY_WALLET` and `STAKING_WALLET` as part of the transfer flow. These addresses can either be wallet addresses or contracts. If any of the addresses belongs to a contract then it may revert from incoming payment. As a result, the error will propagate to the token's contract and revert the transfer.

Shell

```
(bool treasurySuccess,) =  
payable(TREASURY_WALLET).call{value: ethForTreasury}("");  
  
(bool stakingSuccess,) =  
payable(STAKING_WALLET).call{value: ethForStaking}("");
```

Recommendation

The contract should tolerate the potential revert from the underlying contracts when the interaction is part of the main transfer flow. This could be achieved by not allowing set contract addresses or by sending the funds in a non-revertable way.

PVC - Price Volatility Concern

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L870,879
Status	Unresolved

Description

The contract accumulates ETH in the `buybackETHReserve` to perform automatic buybacks when the price falls below the EMA. The `_tryBuyback` function spends the entire reserve in a single transaction without any maximum limit. If the reserve accumulates a large amount of ETH, the contract will swap it all at once for tokens. It is important to note that the price of the token representing it, can be highly volatile. This means that the value of a price volatility swap involving Ether could fluctuate significantly at the triggered point, potentially leading to significant price volatility for the parties involved.

```
Shell
uint256 ethToSpend = buybackETHReserve;
buybackETHReserve = 0;

uniswapRouter.swapExactETHForTokensSupportingFeeOnTransferTokens{value: ethToSpend}(
    0,
    path,
    DEAD,
    block.timestamp
);
```

Recommendation

The contract could ensure that it will not sell more than a reasonable amount of tokens in a single transaction. A suggested implementation could check that the maximum amount should be less than a fixed percentage of the exchange reserves. Hence, the contract will guarantee that it cannot accumulate a huge amount of tokens in order to sell them.

RCS - Redundant Conditional Statements

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L1057
Status	Unresolved

Description

The setBlacklist function checks `if (owner() == address(0))` after the onlyOwner modifier. This check is unreachable before renounce, `owner()` is never `address(0)` so the condition is always false; after renounce, the onlyOwner modifier reverts before execution reaches this line. Such redundancies are common when simple comparisons or checks are performed within conditional statements, leading to redundant operations.

Shell

```
function setBlacklist(address account, bool blacklisted)
external onlyOwner {
    if (owner() == address(0)) revert
    CannotBlacklistAfterRenounce();
    isBlacklisted[account] = blacklisted;
    emit BlacklistUpdated(account, blacklisted);
}
```

Recommendation

The team is advised to review and remove redundant conditional checks that are already enforced by existing access control modifiers, in order to reduce bytecode size and improve code clarity.

SCB - Sell Cooldown Bypass

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L216,431
Status	Unresolved

Description

The contract implements a time-delayed sell restriction using a 30-minute cooldown period to limit repeated sell executions. However, the cooldown accounting is tied to the seller's address rather than the tokens themselves. As a result, a user can distribute tokens across multiple wallets and execute sell transactions from each wallet independently without ever triggering the cooldown restriction, effectively bypassing the intended rate limiting guarantees.

```
Shell
uint256 public constant SELL_COOLDOWN = 30 minutes;

function _transfer(address from, address to, uint256
amount)
...
    if (isSell && !isExcludedFromTax[from]) {
        if (block.timestamp <
lastSellTimestamp[from] + SELL_COOLDOWN) {
            revert SellCooldownActive();
        }
        lastSellTimestamp[from] = block.timestamp;
    }
```

Recommendation

The team is advised to scope cooldown tracking to a mechanism that cannot be trivially circumvented by cycling wallet addresses. This will enforce cooldown semantics at the token level, preserve the safety assumptions behind sell rate limiting, and ensure sell operations cannot be replayed in rapid succession by distributing tokens across multiple addresses.

UPR - Unprotected Price Reading

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L861,909,929,
Status	Unresolved

Description

The contract determines whether a buyback should be triggered and how the EMA price reference should update by relying on spot price quotations derived from the Uniswap V2 pair reserves at the moment of the transaction. These quotations are obtained via `_getCurrentPriceRaw()` which reflects the current state of the pool, not a time-averaged or manipulation-resistant price. As a result, the EMA update and buyback trigger are directly coupled to transient reserve conditions that can change within the same block. An attacker can trade against the VM888/WETH pool to temporarily distort the reserve ratio, causing the EMA to record a manipulated price or triggering an unnecessary buyback. The attacker can then sandwich the resulting buyback swap to extract value from the ETH reserve. Compared to a standard sandwich attack, the impact is amplified because the manipulated price not only worsens execution but also directly affects the contract's internal EMA tracking and autonomous buyback decisions.

```
Shell
function _getCurrentPriceRaw() private view returns
(uint256)
    ...
    return (reserveETH * 1e18) / reserveToken;

function _updateEMA() private {
    uint256 currentPrice = _getCurrentPriceRaw();
    ...
    currentEMA = (EMA_ALPHA * currentPrice +
    EMA_ONE_MINUS_ALPHA * currentEMA) / 1e18;
```

```
function _tryBuyback() private {  
    ...  
    if (currentPrice >= currentEMA) return;
```

Recommendation

Price-dependent protocol decisions such as EMA updates and buyback triggers should not rely on instantaneous spot prices derived from liquidity pool reserves. The team is advised to incorporate a price reading mechanism that does not depend on mutable pool state at the moment of transfer

L03 - Redundant Statements

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L117
Status	Unresolved

Description

Redundant statements are statements that are unnecessary or have no effect on the contract's behavior. These can include declarations of variables or functions that are not used, or assignments to variables that are never used.

As a result, it can make the contract's code harder to read and maintain, and can also increase the contract's size and gas consumption, potentially making it more expensive to deploy and execute.

Shell

```
error TransferAmountExceedsBalance();
```

Recommendation

To avoid redundant statements, it's important to carefully review the contract's code and remove any statements that are unnecessary or not used. This can help to improve the clarity and efficiency of the contract's code.

By removing unnecessary or redundant statements from the contract's code, the clarity and efficiency of the contract will be improved. Additionally, the size and gas consumption will be reduced.

L16 - Validate Variable Setters

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L1057,1064,1069,1014,1020
Status	Unresolved

Description

The contract performs operations on variables that have been configured on user-supplied input. These variables are missing of proper check for the case where a value is zero. This can lead to problems when the contract is executed, as certain actions may not be properly handled when the value is zero.

Shell

```
function setBlacklist(address account, bool blacklisted)
external onlyOwner {
    isBlacklisted[account] = blacklisted;
}
function setExcludedFromTax(address account, bool excluded)
external onlyOwner {
    isExcludedFromTax[account] = excluded;
}
function setExcludedFromLimits(address account, bool
excluded) external onlyOwner {
    isExcludedFromLimits[account] = excluded;
}
function excludeFromReflections(address account) external
onlyOwner

function includeInReflections(address account) external
onlyOwner
```

Recommendation

By adding the proper check, the contract will not allow the variables to be configured with zero value. This will ensure that the contract can handle all possible input values and avoid unexpected behavior or errors. Hence, it can help to prevent the contract from being exploited or operating unexpectedly.

L19 - Stable Compiler Version

Criticality	Minor / Informative
Location	VM888Cyberscope.sol#L2
Status	Unresolved

Description

The `^` symbol indicates that any version of Solidity that is compatible with the specified version (i.e., any version that is a higher minor or patch version) can be used to compile the contract. The version lock is a mechanism that allows the author to specify a minimum version of the Solidity compiler that must be used to compile the contract code. This is useful because it ensures that the contract will be compiled using a version of the compiler that is known to be compatible with the code.

```
Shell  
pragma solidity ^0.8.20;
```

Recommendation

The team is advised to lock the pragma to ensure the stability of the codebase. The locked pragma version ensures that the contract will not be deployed with an unexpected version. An unexpected version may produce vulnerabilities and undiscovered bugs. The compiler should be configured to the lowest version that provides all the required functionality for the codebase. As a result, the project will be compiled in a well-tested LTS (Long Term Support) environment.

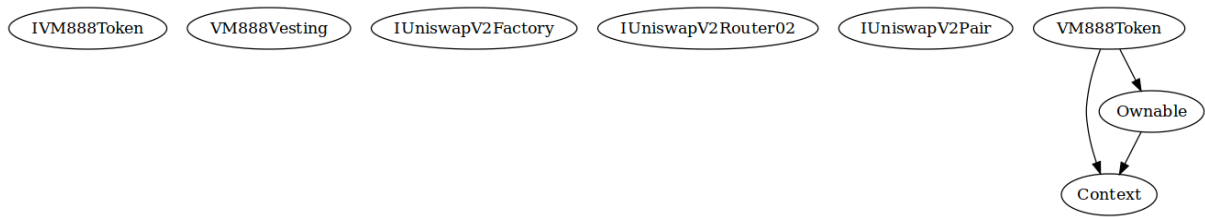
Functions Analysis

Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
IVM888Token	Interface			
	transfer	External	✓	-
	balanceOf	External		-
	getCurrentPrice	External		-
	getEMA	External		-
IUniswapV2Factory	Interface			
	createPair	External	✓	-
	getPair	External		-
IUniswapV2Router02	Interface			
	factory	External		-
	WETH	External		-
	addLiquidityETH	External	Payable	-
	swapExactTokensForETHSupportingFeeOnTransferTokens	External	✓	-
	swapExactETHForTokensSupportingFeeOnTransferTokens	External	Payable	-

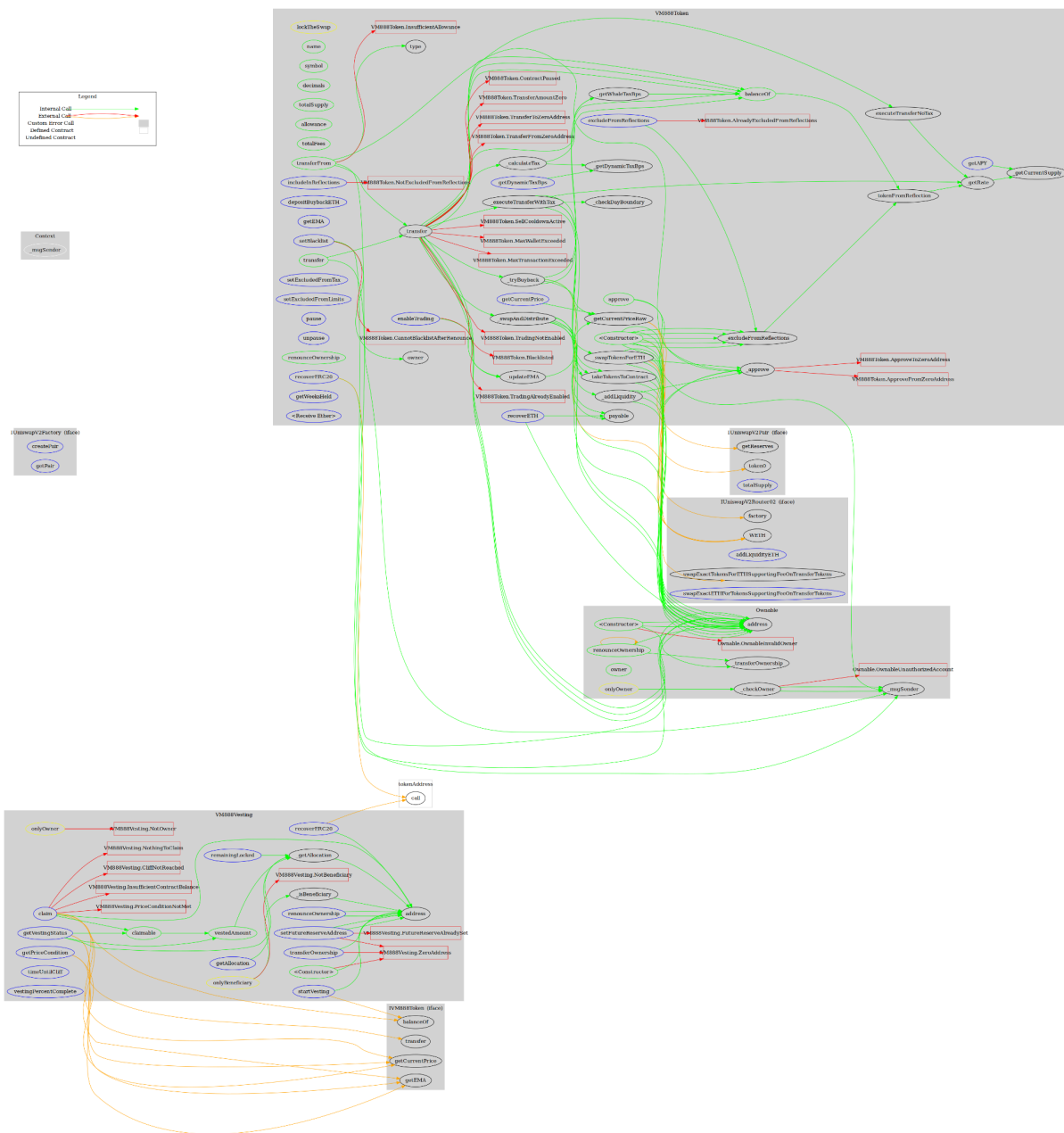
IUniswapV2Pair	Interface			
	getReserves	External		-
	token0	External		-
	totalSupply	External		-
Context	Implementation			
	_msgSender	Internal		
Ownable	Implementation	Context		
		Public	✓	-
	owner	Public		-
	_checkOwner	Internal		
	_transferOwnership	Internal	✓	
	renounceOwnership	Public	✓	onlyOwner
VM888Token	Implementation	Context, Ownable		
		Public	✓	Ownable
	name	Public		-
	symbol	Public		-
	decimals	Public		-
	totalSupply	Public		-
	balanceOf	Public		-
	allowance	Public		-

	totalFees	Public		-
	approve	Public	✓	-
	transfer	Public	✓	-
	transferFrom	Public	✓	-
	_approve	Private	✓	
	_transfer	Private	✓	
	_calculateTax	Private		
	_getDynamicTaxBps	Private		
	_getWhaleTaxBps	Private		

Inheritance Graph



Flow Graph



Summary

VaultedMetrics contract implements a token mechanism. This audit investigates security issues, business logic concerns and potential improvements. There are some functions that can be abused by the owner like stop transactions and massively blacklist addresses. A multi-wallet signing pattern will provide security against potential hacks.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io